Lezione 8 - Chiamate Curl

Introduzione

Nella **lezione precedente** abbiamo analizzato i **sistemi REST** dal punto di vista architetturale, concentrandoci sui principi che regolano la comunicazione tra client e server nel Web moderno.

In particolare, abbiamo visto:

- il significato di **REST** come stile architetturale;
- il ruolo delle **risorse**, identificate tramite URI;
- l'utilizzo dei **metodi HTTP** per operare sulle risorse;
- l'importanza di vincoli come **statelessness**, **cacheability** e **interfaccia uniforme**;
- il **Richardson Maturity Model**, fino al **Livello 3 (HATEOAS)**, che rappresenta la forma più completa e aderente alla filosofia REST.

Abbiamo quindi compreso **come dovrebbe essere progettata un'API RESTful** e quali vantaggi architetturali derivano dal seguire questi principi.

In questa lezione spostiamo l'attenzione dalla **teoria alla pratica**, introducendo le **chiamate cURL**.

Cosa sono le chiamate cURL

cURL è uno strumento a riga di comando che consente di trasferire dati tramite URL.
In termini pratici:

- **permette di comunicare direttamente con i server web utilizzando il terminale, senza passare da un browser o da un'interfaccia grafica.**

Un modo efficace per comprenderne il ruolo è pensare a cURL come a un **browser web senza interfaccia grafica**:
invece di cliccare su pulsanti o compilare moduli, si inviano richieste HTTP scrivendo comandi testuali.
Questa caratteristica lo rende particolarmente adatto a:

- **testare e analizzare il comportamento delle API;**

- **automatizzare operazioni ripetitive;**
- **comprendere in modo esplicito il funzionamento del protocollo HTTP.**

Quindi **cURL** è uno strumento a riga di comando che consente di:

- **inviare richieste HTTP (GET, POST, PUT, PATCH, DELETE)** ;
- **specificare header, parametri e body della richiesta;**
- **interagire direttamente con API RESTful senza utilizzare un browser o un'applicazione client dedicata.**

L'uso di cURL permette di **osservare e comprendere in modo concreto** come avviene la comunicazione HTTP, rendendo espliciti:

- i metodi utilizzati;
- gli endpoint chiamati;
- i dati inviati e ricevuti;
- le risposte del server.

In altre parole:
cURL rappresenta uno strumento fondamentale per testare, studiare e comprendere il funzionamento reale delle API REST, ponendo le basi per l'utilizzo di strumenti più avanzati e per lo sviluppo di client applicativi.

Sintassi di base

La sintassi generale di una chiamata cURL è la seguente:

Shell

```shell
curl [opzioni] [URL]
```

- **URL**: **identifica la risorsa remota con cui si vuole comunicare;**
- **opzioni**: **permettono di modificare il comportamento di cURL**, ad esempio:
  - **specificare il** metodo HTTP **da utilizzare;**
  - **inviare dati nel corpo della richiesta;**
  - **aggiungere** header **personalizzati.**

Ad esempio:

- **per recuperare una risorsa, è sufficiente indicare l'URL;**

- **per inviare dati a un server,**
- si può specificare:
  - **l'opzione `-X`:**
    - **per indicare il metodo HTTP (come `POST` o `PUT`);**
  - **l'opzione `-d`:**
    - **per includere i dati nel body della richiesta.**

Esempi di chiamate cURL

Ottenere tutti gli utenti (GET)

```shell
curl -X GET "https://jsonplaceholder.typicode.com/users"
```

Questa chiamata invia una richiesta **GET** al server e restituisce l'elenco completo degli utenti disponibili.

Ottenere un singolo utente (GET)

```shell
curl -X GET "https://jsonplaceholder.typicode.com/users/1"
```

In questo caso, l'URL identifica una **risorsa specifica** (l'utente con `id = 1`) e il server risponde con i dati relativi a quel singolo utente.

ⓘ Chiamate cURL su Windows

Se si scrive:

```shell
curl [opzioni] [URL]
```

L'output che comparirà su Powershell o il Prompt dei comandi su windows sarà:

```powershell
CategoryInfo : InvalidArgument: (:) [Invoke-WebRequest],
ParameterBindingException + FullyQualifiedErrorId :
```

```
    NamedParameterNotFound,Microsoft.PowerShell.Commands.InvokeWeb
```

Questo comportamento è perfettamente sensato nell'ambiente windows Powershell:
Il comando `curl`, in Windows Powershell, **non è il vero cURL,**
ma piuttosto un alias che punta al cmdlet Powershell:

PowerShell

```
Invoke-WebRequest
```

Il cmdlet `Invoke-WebRequest` **non riconosce l'opzione** `-X`, che invece è propria del vero strumento cURL.
Per questo PowerShell restituisce l'errore:

> *Impossibile trovare un parametro corrispondente al nome 'X'*

Per ovviare a questo problema esistono 2 modalità:
1. Usare il vero cURL(`curl.exe` **)**
Su Windows moderno, cURL è comunque installato, ma va richiamato **esplicitamente**:

PowerShell

```
curl.exe -X GET "https://jsonplaceholder.typicode.com/users"
```

Questo comando:

- bypassa l'alias di PowerShell;
- utilizza il **vero cURL**;
- funziona esattamente come su Linux e macOS.

  2. Usare il cmdlet di Powershell
  L'alternativa è quella di usare il cmdlet di powershell (`Invoke-WebRequest`),
  tuttavia la sintassi cambia:

PowerShell

```
Invoke-WebRequest -Method GET -Uri
"https://jsonplaceholder.typicode.com/users"
```

Oppure, usando l'alias:

PowerShell

```powershell
iwr -Method GET -Uri "https://jsonplaceholder.typicode.com/users"
```

⚠ **Questa non è una chiamata cURL, ma una chiamata PowerShell.**

Esempi di chiamate cURL: operazioni CRUD sugli utenti

Dopo aver visto come effettuare richieste **GET**, analizziamo ora le operazioni di **creazione**, **aggiornamento** e **modifica parziale** di una risorsa, utilizzando i principali metodi HTTP previsti dallo stile REST.

Creare un nuovo utente (POST)

Il metodo POST viene utilizzato per creare una nuova risorsa sul server.
In questo caso, inviamo al server i dati di un nuovo utente in formato JSON.

Il server, se l'operazione va a buon fine, risponderà con:

- **i dati dell'utente appena creato;**
- **un identificatore (`id`) assegnato dal server.**
  Esempio di richiesta:

Shell

```shell
curl -X POST "http://localhost:3000/users" \
-H "Content-Type: application/json; charset=UTF-8" \
-d '{
  "name": "Mario Rossi",
  "username": "mariorossi",
  "email": "mario.rossi@example.com",
  "address": {
    "street": "Via Roma",
    "suite": "App. 1",
    "city": "Milano",
    "zipcode": "20121",
    "geo": {
      "lat": "45.4642",
```

```
14          "lng": "9.1900"
15        }
16      },
17      "phone": "333-1234567",
18      "website": "mariorossi.it",
19      "company": {
20        "name": "Rossi SpA",
21        "catchPhrase": "Costruzioni di qualità",
22        "bs": "realizzazione progetti"
23      }
24    }'
```

Osservazione didattica

POST è **non idempotente**: inviare la stessa richiesta più volte può generare più risorse distinte.

Aggiornare completamente un utente (PUT)

Il metodo PUT viene utilizzato per sostituire interamente una risorsa esistente.
Quando si usa PUT:

- **quando è necessario inviare l'intera rappresentazione della risorsa, anche se solo alcuni campi sono cambiati.**
  Esempio di richiesta

Shell

```
1   curl -X PUT "https://jsonplaceholder.typicode.com/users/1" \
2   -H "Content-Type: application/json; charset=UTF-8" \
3   -d '{
4     "id": 1,
5     "name": "Mario Rossi",
6     "username": "mariorossi",
7     "email": "mario.rossi@example.com",
8     "address": {
9       "street": "Via Roma",
10      "suite": "App. 1",
11      "city": "Milano",
12      "zipcode": "20121",
13      "geo": {
```

```
14          "lat": "45.4642",
15          "lng": "9.1900"
16        }
17      },
18      "phone": "333-1234567",
19      "website": "mariorossi.it",
20      "company": {
21        "name": "Rossi SpA",
22        "catchPhrase": "Costruzioni di qualità",
23        "bs": "realizzazione progetti"
24      }
25    }'
```

Osservazione didattica

PUT è **idempotente**: ripetere la stessa richiesta produce sempre lo stesso stato finale della risorsa.

Aggiornare parzialmente un utente (PATCH)

Il metodo PATCH è pensato per modificare solo alcuni campi di una risorsa, senza dover inviare l'intero oggetto.

È particolarmente utile quando:

- **le risorse sono complesse**
- **o quando si desidera ridurre la quantità di dati trasmessi.**

Esempio di richiesta:

Shell

```
1  curl -X PATCH "https://jsonplaceholder.typicode.com/users/1" \
2  -H "Content-Type: application/json; charset=UTF-8" \
3  -d '{
4    "username": "mariorossi",
5    "email": "mario.rossi@example.com"
6  }'
```

Osservazione didattica

PATCH **non è necessariamente idempotente**: dipende dalla logica implementata dal server.

Il metodo OPTIONS

Oltre ai metodi più comuni, HTTP mette a disposizione il metodo **OPTIONS:**

- **utilizzato per richiedere informazioni sulle capacità del server, senza modificare alcuna risorsa.**

A cosa serve OPTIONS

Una richiesta OPTIONS permette di chiedere al server:

- **quali metodi HTTP sono consentiti su un endpoint;**
- **quali header possono essere inviati;**
- **da quali origini (domini) sono accettate le richieste.**

Questo metodo è fondamentale nel contesto di **CORS** e **viene spesso inviato automaticamente dai browser prima di richieste complesse**.

Esempio di richiesta OPTIONS

Richiesta

HTTP

```http
OPTIONS /api/users HTTP/1.1
Host: api.example.com
```

Risposta del server:

HTTP

```http
HTTP/1.1 200 OK
Allow: GET, POST, PUT, DELETE, OPTIONS
Access-Control-Allow-Origin: *
Access-Control-Allow-Methods: GET, POST, PUT, DELETE
Access-Control-Allow-Headers: Content-Type, Authorization
```

Interpretazione della risposta

Il server sta comunicando che:

- **sull'endpoint** `/api/users` **sono consentiti i metodi** `GET`, `POST`, `PUT`, `DELETE` **e** `OPTIONS`;
- **accetta richieste da qualsiasi dominio (** `*` **);**
- **permette l'uso degli header** `Content-Type` **e** `Authorization`.

ⓘ Come inviare una richiesta OPTIONS con cURL

La sintassi è:

Shell

```shell
curl -X OPTIONS https://jsonplaceholder.typicode.com/users
```

Oppure, in forma equivalente (più esplicita):

Shell

```shell
curl --request OPTIONS
https://jsonplaceholder.typicode.com/users
```

⚠ Perché `curl -X OPTIONS` non mostra nessun output:

Il commando:

Shell

```shell
curl -X OPTIONS https://jsonplaceholder.typicode.com/users
```

Non mostra nessun output di per se, questo perché:

- `OPTIONS` **è un metodo informativo**
  Di conseguenza non modifica lo stato del server e serve a render l'API auto-descrittiva.
  Quindi `OPTIONS`, **è uno strumento di "scoperta" dell'API, non di utilizzo diretto.**
  Per mostrare gli header HTTP possiamo digitare questo commando:

Shell

```shell
curl -i -X OPTIONS
https://jsonplaceholder.typicode.com/users
```

Oppure, ancora meglio:

```shell
curl -I -X OPTIONS
https://jsonplaceholder.typicode.com/users
```

Digitando questo commando l'expected output sarà qualcosa di simile:

```http
HTTP/1.1 204 No Content
Access-Control-Allow-Origin: *
Access-Control-Allow-Methods: GET, POST, PUT, PATCH,
DELETE
Access-Control-Allow-Headers: Content-Type
```

Spiegazione didattica:

- `204 No Content` → risposta valida, ma senza body
- `Access-Control-Allow-Methods` → **metodi HTTP consentiti**
- `Access-Control-Allow-Headers` → header ammessi
- `Access-Control-Allow-Origin` → regole CORS

Nota specifica su JSONPlaceholder

JSONPlaceholder è un servizio **mock**:
non sempre restituisce tutti gli header tipici di un backend reale.

Su un'API reale (Node, Spring, Laravel, ecc.) l'output di `OPTIONS` è molto più esplicito.

Il metodo OPTIONS nella pratica

Nella maggior parte dei casi, **non è necessario inviare manualmente una richiesta OPTIONS**.
OPTIONS è un metodo pensato soprattutto per descrivere le capacità di un endpoint, non per eseguire operazioni sui dati.

Può tuttavia essere utilizzato **esplicitamente** in alcuni contesti specifici:

- **Debug**: **per verificare quali** metodi HTTP **sono consentiti su un determinato endpoint;**
- **Sviluppo**: **per comprendere il comportamento e i limiti di un'**API**;**
- **Tool di test e ispezione**: **per analizzare la configurazione del server (metodi ammessi, header consentiti, policy di sicurezza).**

In altre parole, OPTIONS serve a rispondere alla domanda:

> *"Cosa posso fare su questa risorsa?"*

Confronto con gli altri metodi HTTP

Consideriamo un endpoint `/api/users`:

- `GET /api/users` → **restituisce la lista degli utenti;**
- `POST /api/users` → **crea un nuovo utente;**
- `DELETE /api/users` → **elimina tutti gli utenti;**
- `OPTIONS /api/users` → **non modifica nulla, ma restituisce informazioni su quali operazioni sono permesse.**

> OPTIONS, quindi, **non agisce sui dati**, ma interroga il server sulle sue regole.

Il legame tra OPTIONS e CORS

Il ruolo più importante di OPTIONS emerge nel contesto dei **browser**, in particolare con le **richieste cross-origin**.

Quando un'applicazione web effettua una richiesta che il browser considera "potenzialmente pericolosa" (ad esempio `DELETE`, `PUT`, oppure `POST` con header particolari), **il browser interviene automaticamente inviando una richiesta OPTIONS preliminare, detta preflight request.**

Questo comportamento è **automatico** e fa parte del meccanismo di sicurezza CORS.

Cosa succede realmente nel browser

Supponiamo di scrivere in fetch Data in JavaScript:

JS JavaScript

```javascript
fetch('/api/users', { method: 'DELETE' })
```

Il flusso reale è il seguente:

1. **Il browser rileva che `DELETE` è un metodo sensibile;**
2. **invia automaticamente**:

HTTP

```http
OPTIONS /api/users
```

3. **Il server risponde indicando se `DELETE` è consentito;**
4. **solo se la risposta è positiva, il browser invia la richiesta reale:**

HTTP

```http
DELETE /api/users
```

⚠ **non sei tu a scrivere OPTIONS** in questo caso, Il browser lo fa **per conto tuo.**

Quando il browser invia OPTIONS automaticamente

Alcuni esempi tipici:

- `method: 'GET'`
  → il browser invia **solo GET**
- `method: 'DELETE'`
  → il browser invia **OPTIONS + DELETE**
- `Content-Type: 'application/json'` in una POST
  → il browser invia **OPTIONS + POST**

Questo comportamento:

- **non può essere disabilitato;**
- **non dipende dal codice dell'applicazione;**
- **è una misura di sicurezza obbligatoria.**

Quando si usa OPTIONS manualmente

L’unico caso in cui ha senso scrivere esplicitamente una richiesta OPTIONS è **a scopo di analisi o debug**, ad esempio:

JS JavaScript

```javascript
fetch('/api/users', { method: 'OPTIONS' })
```

Oppure tramite strumenti come `curl`.